

Heaven's Light is Our Guide

Rajshahi University of Engineering & Technology

Department of Computer Science & Engineering



LAB MANUAL

Course No. : CSE 1204

Course Name : Object-Oriented Programming Sessional

Semester : 1st Year Even Semester

Prepared By :	Dr. Md. Shahid Uz Zaman Professor, Department of Computer Science & Engineering (CSE), Rajshahi University of Engineering & Technology (RUET).	Shyla Afroge, Associate Professor, Department of Computer Science & Engineering (CSE), Rajshahi University of Engineering & Technology (RUET).
----------------------	---	---

Object-Oriented Programming Laboratory Guidelines

1. Pre-Lab Instructions for OOP Sessional

- A. **Review the Related Theory**
- B. **Understand the Problem Statement**
 - a. Go through the lab exercise or experiment sheet provided in advance.
 - b. Analyze what the program is expected to do, its inputs, outputs, and logic.
- C. **Write the Algorithm and Flowchart:**
 - a. Prepare an algorithm or pseudocode for the program.
 - b. Draw a flowchart (if applicable) showing program flow and object relationships.
- D. **Plan the Class Design:**
 - a. Identify classes, attributes, methods, and relationships (inheritance, association, composition, etc.).
 - b. Sketch a simple **UML diagram** representing class relationships if required.
- E. **Pre-write the Code (Draft):**
 - a. Write the program in your notebook or on your computer before coming to the lab.
 - b. Check the syntax and structure — ensure it compiles without major errors.
- F. **Bring Required Materials:**
 - a. Lab manual, notebook, pen/pencil, and a storage device (USB drive or GitHub repository access if applicable).
 - b. A copy of your draft program (soft or hard copy).
- G. **Set Up Development Environment:**
 - a. Ensure the required software (e.g., JDK, IDE like Eclipse/NetBeans/IntelliJ, or text editor) is installed on your laptop or familiarized in lab PCs.
 - b. Verify that you can compile and run a basic Java/C++ program.
- H. **Be Prepared for Viva:**
 - a. Be ready to answer theoretical questions related to the lab topic.
 - b. Understand why and how the program uses OOP principles (Encapsulation, Inheritance, Polymorphism, Abstraction).
- I. **Follow Ethical & Safety Rules:**
 - a. Do not copy others' code; every student must write their own.
 - b. Handle lab equipment and computers responsibly.

2. During Lab Session

- A. **Maintain Discipline and Attendance:**
 - Be punctual and attend the lab regularly.
 - Sign the attendance sheet and take assigned seats.
- B. **Follow Instructions Carefully:**
 - Listen to the lab instructor's guidance and adhere to safety and ethical rules.
 - Work individually unless group work is specifically allowed.
- C. **Write the Aim and Program Clearly:**
 - Record the aim, algorithm, and program code neatly in the lab notebook before execution.
- D. **Execute and Debug the Program:**
 - Type and run the program on the computer.

- Debug errors logically and take help from the instructor only after self-attempt.
- E. **Ensure Proper Output:**
 - Verify that the output is correct and matches the expected results.
 - Record sample input and output in the lab notebook.
- F. **Understand the Concept:**
 - Be ready to explain how your program demonstrates OOP principles (Encapsulation, Abstraction, Inheritance, and Polymorphism).
- G. **Avoid Plagiarism:**
 - Do not copy others' code or share your own.
 - Every student's work must reflect their own understanding.
- 2. **Get Lab Sheet Verified:**
 - Show your program and output to the instructor for verification and signature.

3. Post Lab Session

- A. **Prepare a Clean Report:**
 - Write a clean version of the program, algorithm, and output in the lab notebook or manual.
 - Include explanations of results and learning outcomes.
- B. **Answer Viva Questions:**
 - Be prepared to answer conceptual or code-related questions asked by the instructor.
- C. **Submit Lab Records:**
 - Ensure your lab record is completed, verified, and signed before submission.
 - Late submissions may lead to mark deductions.
- D. **Reflect and Revise:**
 - Review errors encountered during the lab and learn how to avoid them.
 - Revise key OOP concepts implemented in the experiment.
- E. **Backup Your Work:**
 - Save a copy of your source codes for future reference and final evaluation

4. Lab Report Writing

Each experiment report should include:

1. **Experiment name & objective**
2. **Theory (short, relevant)**
3. **Code used** (well commented)
4. **Procedure followed**
5. **Observations / output** (LED blink, button press effect, etc.)
6. **Precautions**
7. **Conclusion**

5. General Rules & Precautions

-  Do not install unauthorized software on lab PCs.
-  No mobile usage unless for experiment reference.
-  Maintain a **lab notebook** — checked regularly.

6. Evaluation

♦ 1. Continuous Assessment (75 marks)

Component	Description	Marks Distribution
Lab Performance	Regular attendance, punctuality, active participation, and discipline in lab activities.	15%
Program Implementation	Correctness, logic, use of OOP principles (encapsulation, inheritance, polymorphism, abstraction), efficiency, and code readability.	20%
Viva Voce (Per Lab)	Understanding of theory behind the experiment, ability to explain code, debugging approach, and problem-solving skills.	10%
Lab Report / Notebook	Neat presentation, completeness (aim, algorithm, flowchart, code, output, and result discussion), and timely submission.	10%
Quiz	Quiz Test on Total Lab experiment	20

Lab Curriculum**Table-1: CO-PO mapping for CSE 3106**

CO	CO Statement	PO	Delivery Method	Assessments
CO1	Solve real life problems from an object oriented perspective. Fundamentals of OOP, Classes and Objects, Constructors and Destructors, Inheritance, Polymorphism	PO2	• Lectures and Tutorials • Hands-on Lab Sessions • Case Studies	• Project Based Assignments • Lab Experiments
CO2	Identify and define aspects of object-oriented design incorporating UML.	PO3	• Lectures and Tutorials • Hands-on Lab Sessions • Case Studies	• Lab Experiments • Practical Lab Tests
CO3	Perform individually or as a part of a team while developing object-oriented software.	PO9	• Lectures and Tutorials • Hands-on Lab Sessions • Case Studies	• Lab Experiments • Practical Lab Tests
CO4	Design and build innovative software solutions using object-oriented concepts for real life problems.	PO4	• Lectures and Tutorials • Hands-on Lab Sessions • Case Studies	• Lab Experiments • Practical Lab Tests

Table-2: PO statements

PO1	Engineering knowledge: Apply knowledge of mathematics, natural science, engineering fundamentals and an engineering specialization respectively to the solution of complex engineering problems.
PO2	Problem analysis: Identify, formulate, research literature and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences and engineering sciences.
PO3	Design/development of solutions: Design solutions for complex engineering problems and design systems, components or processes that meet specified needs with appropriate consideration for public health and safety, cultural, societal, and environmental considerations.
PO4	Investigation: Conduct investigations of complex problems using research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of information to provide valid conclusions.
PO5	Modern tool usage: Create, select and apply appropriate techniques, resources, and modern engineering and IT tools, including prediction and modelling, to complex engineering problems, with an understanding of the limitations.
PO6	The engineer and society: Apply reasoning informed by contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to professional engineering practice and solutions to complex engineering problems.
PO7	Environment and sustainability: Understand and evaluate the sustainability and impact of professional engineering work in the solution of complex engineering problems in societal and environmental contexts.
PO8	Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of engineering practice.
PO9	Individual work and teamwork: Function effectively as an individual, and as a member or leader in diverse teams and in multi-disciplinary settings.
PO10	Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
PO11	Project management and finance: Demonstrate knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
PO12	Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

List of Experiments

Lab 1	
Problem 1	Problem statement: Write a circle class with following data members and methods. The structure of the circle class is <pre> class Circle { private: int radius; float area;</pre>

	<pre>// Write methods. }; Now do the following: i) Initialize radius of 3 circles ii) Find area of all of them iii) Find the total area</pre>
Problem 2	Problem statement: Write an Account class with 2 data members and methods. The structure of the Account class is <pre>class Account { int number; int amount; //Write methods };</pre> Now do the following i) Initialize 5 accounts ii) Deposit money to an account iii) Withdrawal money from an account iv) Transfer money from one account to another
Problem 3	Problem statement: Write a Triangle class with 3 edges as data members and methods. The structure of the Tringle class is: <pre>class Triangle { int edge1; int edge2; int edge3; float area; // write methods; }</pre> Now do the following i) Initialize edges of a triangle ii) Find area of the triangle iii) Check whether the 3 edges form a triangle
Problem 4	Problem statement: Want to feel better, have more energy and even add years to your life? Just exercise. Joining a gym can help you stay motivated to exercise consistently. This is a great way to build muscle, lose weight, lower blood pressure, boost mental focus, and more. Over time, you can look better, feel better, and accomplish things you never thought possible! It is important to know your BMI (Body Mass Index) for the good health. (for more read below) Write a C/C++ program to the process the Gym data using the following constraints: i. Store ID, Height and Weight of each member ii. A member can be added/removed/updated iii. The program should be menu operated iv. Define a structure with data members ID, Height and Weight. v. Calculate average Height of the members vi. Calculate average Weight of the members vii. Calculate Max Height and Weight viii. Calculate Min Height and Weight

ix. Display BMI classification of a given member (use following table)

a. The structure of the Member is

```
Class Member {
    //declare data members;
    // write methods
};
```

b. BMI Calculation

$$\text{BMI} = (\text{Weight in Kg}) / (\text{Height in m})^2$$

Classification	BMI range - kg/m ²
Severe Thinness	< 16
Moderate Thinness	16 - 17
Mild Thinness	17 - 18.5
Normal	18.5 - 25
Overweight	25 - 30
Obese Class I	30 - 35
Obese Class II	35 - 40
Obese Class III	> 40

c. The menu look like

***** Main Menu *****

1. Add Member
2. Update Member
3. Remove Member
4. Max Height & Weight
5. Min Height and Weight
6. Average Height and Weight
7. BMI Classification
8. Exit

Enter Your option(1-8):_

Lab 2

Problem
1

Problem statement: Use the following class Test with the data members and methods

```
Class Test {
```

```
Private:
```

```
Data member x;
```

```
Data member y;
```

```
Data member z;
```

```
//write methods;
```

```
};
```

Now write and necessary codes in class and **main()** do the following

- i) Initialize private data members **x** and **y** to 0 when empty constructor is called
- ii) Initialize private data members **x** and **y** using parameterized constructor is called
- iii) Initialize private data members **x** and **y** from another object using copy constructor

	iv) The data member z keeps track of total objects created v) Write a method to initialize x and y vi) Write a method to display data member z only vii) Write a method to display x,y and z where their values can't be changed viii) Create five objects ix) Find the sum of x x) Find the object number whose y value is maximum Write destructor																																																																																								
Problem 2	Problem statement: Write a class A with two data member a and b . Now set the values of a and b of an object obj1 by a setter method SetData() and copy the values of a and b of obj1 to another object obj2 by using copy constructor.																																																																																								
Lab 3																																																																																									
Problem 1	Problem statement: You have the create an inheritance among Father-->Son-->GrandSon class. The fatherclass has the following data members class Father { private: int money; protected: int gold; public: int land; }; Now write the Sonand GrandSon classes with private/protected/publicaccess modifier and do the following: i) Try to access money, goldand landfrom Sonclass ii) Try to access money, goldand landfrom GrandSon class iii) Find the values of money, gold and land when different access modifier is used in the following table iv) Display the sum of money, gold and land in Grandsonclass. <table><tr><th colspan="2">Class</th><th colspan="3">In Son class</th><th colspan="3">In GrandSon class</th></tr><tr><th>Son</th><th>GrandSon</th><th>money</th><th>gold</th><th>land</th><th>money</th><th>gold</th><th>land</th></tr><tr><td>public</td><td>public</td><td>?</td><td>?</td><td>?</td><td>?</td><td>?</td><td>?</td></tr><tr><td>protected</td><td>public</td><td>?</td><td>?</td><td>?</td><td>?</td><td>?</td><td>?</td></tr><tr><td>private</td><td>public</td><td>?</td><td>?</td><td>?</td><td>?</td><td>?</td><td>?</td></tr><tr><td>public</td><td>protected</td><td>?</td><td>?</td><td>?</td><td>?</td><td>?</td><td>?</td></tr><tr><td>protected</td><td>protected</td><td>?</td><td>?</td><td>?</td><td>?</td><td>?</td><td>?</td></tr><tr><td>private</td><td>protected</td><td>?</td><td>?</td><td>?</td><td>?</td><td>?</td><td>?</td></tr><tr><td>public</td><td>private</td><td>?</td><td>?</td><td>?</td><td>?</td><td>?</td><td>?</td></tr><tr><td>protected</td><td>private</td><td>?</td><td>?</td><td>?</td><td>?</td><td>?</td><td>?</td></tr><tr><td>private</td><td>private</td><td>?</td><td>?</td><td>?</td><td>?</td><td>?</td><td>?</td></tr></table>	Class		In Son class			In GrandSon class			Son	GrandSon	money	gold	land	money	gold	land	public	public	?	?	?	?	?	?	protected	public	?	?	?	?	?	?	private	public	?	?	?	?	?	?	public	protected	?	?	?	?	?	?	protected	protected	?	?	?	?	?	?	private	protected	?	?	?	?	?	?	public	private	?	?	?	?	?	?	protected	private	?	?	?	?	?	?	private	private	?	?	?	?	?	?
Class		In Son class			In GrandSon class																																																																																				
Son	GrandSon	money	gold	land	money	gold	land																																																																																		
public	public	?	?	?	?	?	?																																																																																		
protected	public	?	?	?	?	?	?																																																																																		
private	public	?	?	?	?	?	?																																																																																		
public	protected	?	?	?	?	?	?																																																																																		
protected	protected	?	?	?	?	?	?																																																																																		
private	protected	?	?	?	?	?	?																																																																																		
public	private	?	?	?	?	?	?																																																																																		
protected	private	?	?	?	?	?	?																																																																																		
private	private	?	?	?	?	?	?																																																																																		
Problem 2	[Types of Inheritance] Problem statement: Learn and Test different types of inheritance in C++. In each inheritance draw the class diagram with class chain and try to access the data members of bases classes from child classes. i. Single Inheritance																																																																																								

class A{ private: int x; protected: int y; public: int z; }	class B:public A{ //write public method to //access x,y & z }	int main(){ B b; //call methods of class B return 0; }
--	--	---

ii. Multi-Level Inheritance

class A{ private: int x; protected: int y; public: int z; }	Class B: public A{ }	class C:public B{ //write public method to access x,y & z }	int main(){ C c; //call methods of class C return 0 }
---	----------------------------	---	---

iii. Multiple Inheritance

class A{ private: int x; protected: int y; public: int z; }	class B{ private: int p; protected: int q; public: int r; }	class C: public A, Public B{ //write public method to access x,y,z,p,q & r. }	int main(){ C c; //call methods of class C return 0; }
--	--	--	---

iv. Hierarchical Inheritance

class A{ private: int x; protected: int y; public: int z; }	class B: public A { //write public method to access x,y & z }	class C:public A { //write method public to access x,y & z }	int main(){ B b; C c; //call methods of class B & C return 0
--	---	---	--

v. Hybrid Inheritance

	<pre>class A{ private: int x; protected: int y; public: int z; }</pre>	<pre>class B: public A { }</pre>	<pre>class C: public A { }</pre>	<pre>class D: public B, public C { //write public method to access x,y & z }</pre>	<pre>int main(){ D d; //call methods of class D return 0 }</pre>											
Problem 3	[Constructor & Destructor in inheritance] Problem statement: Write the constructors & destructors for different types of inheritance are given as follows. Also follow and write the sequence of their execution. i. Single inheritance <table><tr><td><pre>class A{ private: int ax; public: //write constructor to initialize ax; //write destructor }</pre></td><td><pre>class B: public A{ Private: Int bx; public: // write constructor to initialize bx; //write methods to sum ax and bx //write destructor }</pre></td><td><pre>int main(){ B b; //call methods of B; return 0; }</pre></td></tr></table> ii. Multi-level inheritance <table><tr><td><pre>private: int ax; public: //write constructor to initialize ax //Write destructor }</pre></td><td><pre>class B:public A { private: int bx; public: //write constructor to initialize bx //Write destructor }</pre></td><td><pre>class C:public B { private: int cx; public: //write constructor to initialize cx //Write method to sum ax, bx and cx //Write destructor }</pre></td><td><pre>int main(){ C c; //call //methods of //class C return 0 }</pre></td></tr></table> iii. Multiple inheritance <table><tr><td><pre>private: int ax; public: //write constructor to initialize ax //Write destructor }</pre></td><td><pre>class B{ private: int bx; public: //write constructor to initialize bx //Write destructor }</pre></td><td><pre>class C:public A, Public B{ private: int cx; public: //write constructor to initialize cx //Write method to sum ax, bx and cx //Write destructor }</pre></td><td><pre>int main(){ C c; //call //methods of //class C return 0 }</pre></td></tr></table>					<pre>class A{ private: int ax; public: //write constructor to initialize ax; //write destructor }</pre>	<pre>class B: public A{ Private: Int bx; public: // write constructor to initialize bx; //write methods to sum ax and bx //write destructor }</pre>	<pre>int main(){ B b; //call methods of B; return 0; }</pre>	<pre>private: int ax; public: //write constructor to initialize ax //Write destructor }</pre>	<pre>class B:public A { private: int bx; public: //write constructor to initialize bx //Write destructor }</pre>	<pre>class C:public B { private: int cx; public: //write constructor to initialize cx //Write method to sum ax, bx and cx //Write destructor }</pre>	<pre>int main(){ C c; //call //methods of //class C return 0 }</pre>	<pre>private: int ax; public: //write constructor to initialize ax //Write destructor }</pre>	<pre>class B{ private: int bx; public: //write constructor to initialize bx //Write destructor }</pre>	<pre>class C:public A, Public B{ private: int cx; public: //write constructor to initialize cx //Write method to sum ax, bx and cx //Write destructor }</pre>	<pre>int main(){ C c; //call //methods of //class C return 0 }</pre>
<pre>class A{ private: int ax; public: //write constructor to initialize ax; //write destructor }</pre>	<pre>class B: public A{ Private: Int bx; public: // write constructor to initialize bx; //write methods to sum ax and bx //write destructor }</pre>	<pre>int main(){ B b; //call methods of B; return 0; }</pre>														
<pre>private: int ax; public: //write constructor to initialize ax //Write destructor }</pre>	<pre>class B:public A { private: int bx; public: //write constructor to initialize bx //Write destructor }</pre>	<pre>class C:public B { private: int cx; public: //write constructor to initialize cx //Write method to sum ax, bx and cx //Write destructor }</pre>	<pre>int main(){ C c; //call //methods of //class C return 0 }</pre>													
<pre>private: int ax; public: //write constructor to initialize ax //Write destructor }</pre>	<pre>class B{ private: int bx; public: //write constructor to initialize bx //Write destructor }</pre>	<pre>class C:public A, Public B{ private: int cx; public: //write constructor to initialize cx //Write method to sum ax, bx and cx //Write destructor }</pre>	<pre>int main(){ C c; //call //methods of //class C return 0 }</pre>													

	iv. Hierarchical inheritance				
	<pre>class A{ private: int ax; public: //write constructor to initialize ax //Write destructor }</pre>	<pre>class B:public A { private: int bx; public: //write constructor to initialize bx //Write destructor }</pre>	<pre>class C:public A { private: int cx; public: //write constructor to initialize cx //Write method to sum ax, bx and cx //Write destructor }</pre>	<pre>int main(){ B b; C c; //call //methods of //class B & C return 0 }</pre>	
	v. Hybrid (Diamond) inheritance [virtual class]				
	<pre>class A{ private: int ax; public: //write constructo r to initialize ax //Write destructor }</pre>	<pre>class B:public A { private: int bx; public: //write constructo r to initialize bx //Write destructor }</pre>	<pre>class C:public A { private: int cx; public: //write constructor to initialize cx //Write destructor }</pre>	<pre>class D:public B, public C { private: int dx; public: //write constructor to initialize dx //Write method to sum ax, bx cx and dx //Write destructor }</pre>	<pre>int main(){ D d; //call //methods of //class D return 0 }</pre>
Lab 4					
Problem 1	[Method/ Function overloading] Problem statement: Define a class Test where overload a method Sum() to sum numbers sent from main() function. class Test{ public: //overload Sum() method according to the requirements of main () method. } int main(){ Test t; t.Sum(10); // Return 10; t.Sum(10,20); // Return 30 t.Sum(5.7,20); // Return 25.7 t.Sum(10, 2.6); // Return 12.6 t.Sum(10.5,20.5); // Return 31 }				
Problem 2	[Operator Overloading]				

	Problem statement: Suppose in a AC circuit, there are 3 impedances $z_1=3+j4$, $z_2=4-j3$ and $z_3=j6$ are connected in parallel. Now find the current in the circuit if input voltage is $100+j50$. Implement operator overloading concept for your calculation. Use class Circuit and initialize the impedance values (real & img) by a constructor. <pre> class circuit{ private: int real; int img; public: //write constructor; //write operator overloaded method; //write a display method to display real and img }; int main() { circuit z1(3,4); // write required statement to find the current. } </pre>		
Problem 3	[Unary Operator Overloading] Problem statement: Write a program using unary operator overloading to control the sound of a TV remote. If '+' button is pressed then sound increases and if '-' button pressed then sound decreases <table border="1"> <tr> <td> <pre> Class Remote { Int sound; //write your code; } </pre> </td><td> <pre> int main(){ Remote rs; ++rs; //sound++ when '+' button is pressed ++rs; --rs //sound-- when '-' button is pressed. Display(); //display the value of sound return 0;} </pre> </td></tr> </table>	<pre> Class Remote { Int sound; //write your code; } </pre>	<pre> int main(){ Remote rs; ++rs; //sound++ when '+' button is pressed ++rs; --rs //sound-- when '-' button is pressed. Display(); //display the value of sound return 0;} </pre>
<pre> Class Remote { Int sound; //write your code; } </pre>	<pre> int main(){ Remote rs; ++rs; //sound++ when '+' button is pressed ++rs; --rs //sound-- when '-' button is pressed. Display(); //display the value of sound return 0;} </pre>		
Problem 4	[Friend Function] Problem statement: Suppose your money is deposited in two banks through their private data members money1 and money2 respectively. Now based on the statements in main() function, write a program to calculate your total money using a friend function Sum(). <pre> class Bank1 { private: float money1; public: //write your codes }; class Bank2 { private: float money2; public: //write your codes; //write the body of friend functions int main() { </pre>		

	<pre> Bank1 b1(2000); Bank2 b2(4000); Sum (b1, b2); } </pre>	
Lab 5		
Problem 1	[Method/Function Overriding] Problem statement: Write a class A with a method Print() and a derived class B with method Print() overloaded. Now observe the output when following statements are written in the main() function	
	<pre> class A{ public: void Print(){ cout<<"Inside Print() of class A"<<endl; } }; class B:public A{ public: void Print(){ cout<<"Inside Print() of class B"<<endl; } }; </pre>	Write Statements inside main() <pre> i) A a; a.Print(); ii) B b; b.Print(); iii) A a; A *p; p=&a; p->Print(); iv) B b; A *p; p=&b; p->Print(); Repeat i)-iv) after writing virtual in front of void Print() </pre>
Problem 2	[Pure Virtual Function] Problem statement: Modify the class defined in Topic 3 executes the following statements i)-iv) and observe the output:	
	<pre> class A{ public: virtual void Print()=0; } class B:public A{ public: void Print(){ cout<<"Inside Print() of class B"<<endl; } }; </pre>	Write Statements inside main() <pre> iii) A a; a.Print(); iv) B b; b.Print(); iii) A a; A *p; p=&a; p->Print(); iv) B b; A *p; p=&b; p->Print(); </pre>

Problem 3	[Exception Handling] <i>Problem Statement:</i> In the following input i is the index of array ax[]. The program prints ax[i]. Then write catch block if i is out of range of ax[]. Write three catch blocks to fulfill the purpose i) a catch block receives the value of i ii) a catch block receives string "Out of Range Error" iii) a default catch() if above two catch block doesn't match <pre> #include <iostream> using namespace std; int main() { int i; int ax[5]={10,20,60,40,30}; cout<<"enter index:"; cin>>i; cout<<"ax["<<i<<"]="<<ax[i]<<endl; } </pre>
Problem 4	Problem statement: Declare a STL array object ax with 6 elements and do the following: i) Assign 10,60,30,70,20 to ax using single statement ii) Print third element of ax using at() function iii) Print first element of ax using front() function iv) Print last element of ax using back() function v) Fill the elements of ax using fill() function vi) Test whether ax is empty or not using empty() function vii) Print size of ax viii) Print maximum size of ax using max_size() function ix) Print address of first element of ax using begin() function x) Print address of last element of ax using end() function <pre> #include <iostream> #include <bits/stdc++.h> using namespace std; int main(){ array<int,6>ax; //write statements } </pre>
Problem 5	[STL: pair class] Problem statement: Define a pair class object px with int and string elements. Write statements to do the following i) Assign 10 to int and "Rajshahi" to px using make_pair() function ii) Print int data member by first iii) Print string data member by second iv) Modify first data member to 20 using get<>() function v) Declare another pair bx and assign values to bx and swap it with ax

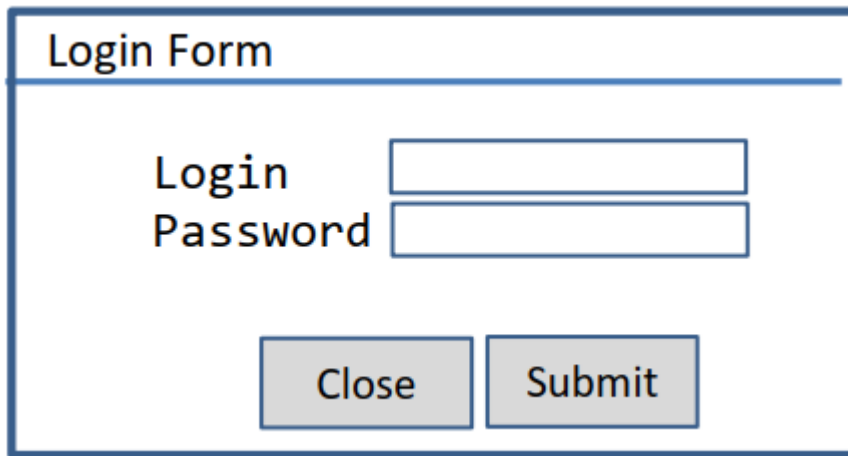
	<pre>#include <iostream> #include <bits/stdc++.h> using namespace std; int main(){ pair<int,string>px; //write statements }</pre>
Problem 6	[STL: tuple class] Problem statement: Define a tuple class object tx with int.string and double elements. Write statements to do the following i) Assign <100,"Kamal",3.5> to tx using make_tuple() function ii) Print int data member by get() function iii) Print string data member by get() function iv) Print double data member by get() function v) Modify third data member to 3.7 using get<>() function vi) Declare another tuple bx and assign values to bx and swap it with ax <pre>#include <iostream> #include <bits/stdc++.h> using namespace std; int main(){ tuple<int,string,double>tx; //write statements }</pre>
Lab 6	
Problem 1	[STL: stack class] Problem statement: Write a menu program to create and manipulate Stack using stack class and perform the following operations using the specified method. i) use push() method to push data ii) use pop() method to pop data iii) use top() method to display top element iv) use empty() method to check whether stack is empty or not <pre>** Stack ** 1. Push 2. Pop 3. Display() 4. Exit Enter your option:</pre>
Problem 2	[STL: queue class] Problem statement: Write a menu program to create and manipulate Queue using queue class. Perform the following operations using the specified method. i) use push() method to enqueue data ii) use pop() method to dequeue data iii) use front() method to display front element iv) use back() method to display rear element v) use empty() method to check whether queue is empty or not

	<pre> ** Queue ** 1. Enqueue 2. Dequeue 3. Display Front 4. Display Rear 5. Exit Enter your option: </pre>			
Problem 3	[STL: vector class] Problem statement: Write a menu program to create and manipulate linked list using vector class and use the following methods i) push_back() ii) push_front() iii) begin() iv) end() v) pop_back() vi) pop_front() vii) insert() viii) erase() [STL: vector class]: Add more feature to the menu of Topic 3. <pre> ** Linked List ** 1. Insert 2. Delete 3. Update 4. Search 5. Exit Enter your option: </pre>			
Lab 7				
Problem 1	[UML] Problem statement: Draw the UML diagram for the following class <pre> class Test{ private: int x; protected: int y; public: int z; public: void SetData(int a,int b,int c){ x=a;y=b;z=c; } float getAverage(){ Return (x+y+z)/3.0; } } </pre>			
Problem 2	[UML: Aggregation] Problem statement: Create two classes Doctor and Patient to store the information of doctors and patients respectively. Now write codes to create association between these two classes by aggregation. [note: if Patient class is destroyed then Doctor class is still exist] <table><tr><td><pre> class Doctor{ //write your code } </pre></td><td><pre> class Patient{ //write your code } </pre></td><td><pre> int main(){ //write your code } </pre></td></tr></table>	<pre> class Doctor{ //write your code } </pre>	<pre> class Patient{ //write your code } </pre>	<pre> int main(){ //write your code } </pre>
<pre> class Doctor{ //write your code } </pre>	<pre> class Patient{ //write your code } </pre>	<pre> int main(){ //write your code } </pre>		

Problem 3	[UML: Composition] Problem statement: Create two classes Doctor and Patient to store the information of doctors and patients respectively. Now write codes to create association between these two classes by composition. [note: if Doctor class is destroyed then Patient class is also destroyed] <table><tr><td><pre>class Doctor{ //write your code }</pre></td><td><pre>class Patient{ //write your code }</pre></td><td><pre>int main(){ //write your code }</pre></td></tr></table>	<pre>class Doctor{ //write your code }</pre>	<pre>class Patient{ //write your code }</pre>	<pre>int main(){ //write your code }</pre>
<pre>class Doctor{ //write your code }</pre>	<pre>class Patient{ //write your code }</pre>	<pre>int main(){ //write your code }</pre>		
Problem 4	[java Programming Basics] <i>Problem Statement:</i> Write java programs to do the following i) Display your name and address in two lines ii) Take two integers as inputs and prints the bigger one iii) Input a series of 10 numbers into an array ax[] and finds the largest, smallest and average of these numbers. Also write code for searching and sorting.			
Problem 5	[java Programming private method] <i>Problem Statement:</i> Verify that private methods can be used only inside the class but it cannot be used from outside the class.			
Problem 6	[java Programming static method/member] <i>Problem Statement:</i> Verify that static methods can access static method/member but cannot access to non-static method/member.			
Lab 8				
Problem 1	[ArrayList class]: Create an object of ArrayList class and implement the following methods: i) add() ii) get() iii) set() iv) remove() v) size() vi) toString() vii) sort() viii) clear()			
Problem 2	[Anonymous Object]: Write a method MidPoint() that accepts two Point objects and return their mid-point anonymously .			
Problem 3	[inner class]: Suppose B is the inner class of A and ax and bx are the private data members of A and B respectively. Now write a method to sum ax and bx using the object of class B.			
Problem 4	[inheritance: use of super keyword] For the following Java program write i) to display x of class A in class B ii) statement to call getX() of class A in class B iii) to call parameterized constructor of in class B			

	<pre>class A{ int x; public A(){ x=0; } public A(int x){ this.x=x; } public int getX(){ return(x+10); } class B extends A{ int x=20; public int getX(){ return(x+10); } } public class First{ public static void main(String[] args) { }</pre>	
Problem 5	[interface in Java] Write three interfaces in Java namely AI, BI and CI with the following activities: i) Declare method PrintA() in AI ii) Declare method PrintB() in BI iii) Declare method PrintC() in CI iv) write body of PrintA() in class A v) write body of PrintB() in class B which is inherited from class A vi) write body of PrintC() in class C which is inherited from class B vii) Create object of class C in main() of First class and invoke PrintA(), PrintB() and PrintC()	
Lab 9		
Problem 1	[inheritance] <i>Problem Statement:</i> Implement the following relation diagram into java program	
<pre>graph BT C1[class] -- extends --> C2[class] C3[class] -- implements --> I1[interface] C4[class] -- extends --> I2[interface]</pre> <pre>graph BT C[class] -- implements --> I1[interface] C -- implements --> I2[interface] I3[interface] -- extends --> I4[interface] I3 -- extends --> I5[interface]</pre> Multiple Inheritance in Java		
Problem 2	[Singleton class] <i>Problem Statement:</i> Write a Java program to create and operate a singleton class.	
Problem 3	[Java Multithreading] <i>Problem Statement:</i> Write a program to run three applications i) Word Processor ii) Music Player and iii) Real-time clock simultaneously. Use run() method of Thread class or implements runnable interface.	
Lab 10		
Problem 1	[Swing in Java]	

Problem Statement: Based on the swing components above, design the following frame and perform login functionality using Java Swing **ActionListener** interface. Displays appropriate messages when required using **showMessageDialog** box.



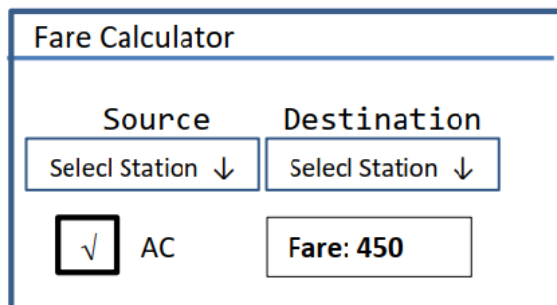
The image shows a Java Swing window titled "Login Form". Inside the window, there are two text labels, "Login" and "Password", positioned to the left of two empty rectangular text input fields. Below these fields, there are two rectangular buttons: "Close" on the left and "Submit" on the right.

Problem
2

[Fare Calculator using Java Swing]

Problem Statement: The following java swing application calculates and displays the fare between two railway stations using Java **ItemListener** and **ActionListener** interfaces. There are two comboboxes for source and destination stations and a checkbox for seat type {AC/Non-AC}. Use the following distance table to calculate the normal (non-ac) fare. Assume fare of Tk 3 for each kilometer and also add 50% more fare if AC seat is selected:

	Rajshahi	Iswardi	Ullapara	Tangail	Dhaka
Rajshahi	0	50	100	150	230



The image shows a Java Swing window titled "Fare Calculator". Inside the window, there are two labels, "Source" and "Destination", each above a combobox. Both comboboxes display "Selected Station" with a downward arrow. Below the "Source" combobox, there is a checked checkbox followed by the text "AC". To the right of this, there is a rectangular box displaying the text "Fare: 450".

Problem
3

Create any application using Java Swing implementation **ActionListener** and **ItemListener** interfaces.